

課題 1: 細分割曲面の実装

氏名：南條佑太
所属・学年：学際科学科 3 年

2026 年 6 月 1 日

1 Loop 細分割曲面

1.1 4-to-1 分割の新しい 4 つの面を submesh に作成するコード

```
1
2 // submesh faces
3 for (auto& fc : mesh_>faces()) {
4     //
5     // ここに 4-to-1 分割の新しい 4 つの面を 4 submesh に作成するコードを追加
6     auto he0 = fc->halfedge(0);
7     auto he1 = fc->halfedge(1);
8     auto he2 = fc->halfedge(2);
9
10    auto v0 = he0->vertex();
11    auto v1 = he1->vertex();
12    auto v2 = he2->vertex();
13
14    auto e0 = he0->edge();
15    auto e1 = he1->edge();
16    auto e2 = he2->edge();
17
18    auto ev0 = even_[v0->id()];
19    auto ev1 = even_[v1->id()];
20    auto ev2 = even_[v2->id()];
21
```

```

22     auto od0 = odd_[e0->id()];
23     auto od1 = odd_[e1->id()];
24     auto od2 = odd_[e2->id()];
25
26     submesh_->addTriangle(ev0, od0, od2);
27     submesh_->addTriangle(ev1, od1, od0);
28     submesh_->addTriangle(ev2, od2, od1);
29     submesh_->addTriangle(od0, od1, od2);
30     //
31
32 }
33 };

```

1.2 プログラムの説明

Loop 細分割において、元のメッシュの1つの三角形の面を、4つの小さな三角形に分割し、新しいサブメッシュに登録する処理である。面のハーフエッジを取得して、頂点とエッジを取得する。そして $ev0ev1ev2, od0od1od2$ のポインタを取得して、新しい4つの三角形を反時計回りの順番で頂点を渡して作成した。

1.3 even vertex の頂点位置の計算コード

```

1  for (auto& vt : mesh_->vertices()) {
2      Eigen::Vector3d p;
3      //
4      // ここに even vertex の頂点位置の計算コードを追加
5      Eigen::Vector3d v = vt->point();
6      Eigen::Vector3d sum_vi = Eigen::Vector3d::Zero();
7      int n = 0;
8
9      VertexLCirculator vcirc(vt);
10     std::shared_ptr<VertexL> vv = vcirc.beginVertexL();
11     do {
12         sum_vi += vv->point();
13         n++;
14         vv = vcirc.nextVertexL();
15     } while ((vv != vcirc.firstVertexL()) && (vv != nullptr));
16     double b = beta(n);
17

```

```

18     p = (1.0 - (double)n * b) * v + b * sum_vi;
19
20
21     //
22
23     even_[vt->id()]->setPoint(p);
24 }

```

1.4 プログラムの説明

既存の頂点が細分割後に移動する新しい座標を計算し、更新する処理である。対象となる頂点の周りの頂点の値をたし、またその接続数を、それらを元に更新式を計算して元の頂点の座標をその値に更新している。

1.5 odd vertex の頂点位置の計算コード

```

1
2     // odd vertex points
3     for (auto ed : mesh_->edges()) {
4         Eigen::Vector3d p;
5         //
6         // ここにodd vertex の頂点位置の計算コードを追加
7         auto he1 = ed->lhe();
8         auto he2 = ed->rhe();
9
10        Eigen::Vector3d v0 = he1->vertex()->point();
11        Eigen::Vector3d v1 = he2->vertex()->point();
12
13        Eigen::Vector3d v2 = he1->next()->vertex()->point();
14        Eigen::Vector3d v3 = he2->next()->vertex()->point();
15
16        p = LOOP_MASK_38 * (v0 + v1) + LOOP_MASK_18 * (v2 + v3);
17
18        odd_[ed->id()]->setPoint(p);
19
20        //
21
22        odd_[ed->id()]->setPoint(p);
23

```

1.6 プログラムの説明

エッジ上に新しく生成される奇数頂点の座標を計算する処理である。エッジの端点の座標の値と、エッジの向かいにある頂点の座標を `-i->next()` `-i->vertex()` `-i->point()` で取得して、Loop 細分割のルールに基づき計算して新しい座標を更新した。

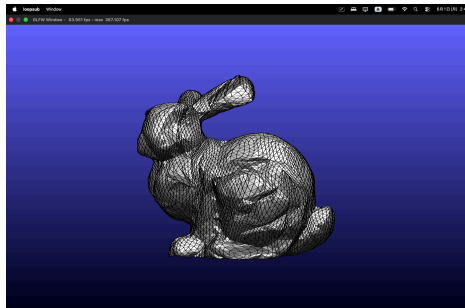


図 1 図の説明

1 回および 2 回細分割した後の頂点数・面数: 1 回目 v 1994 f 3984 2 回目 v 7970 f 15936

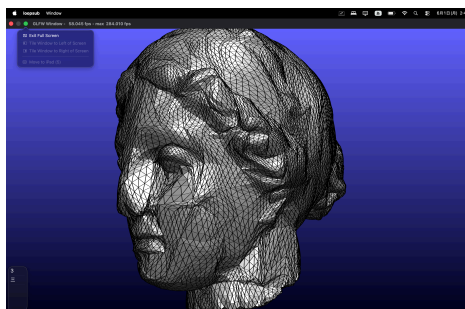


図 2 図の説明

1 回および 2 回細分割した後の頂点数・面数: 1 回目 v 3994 f 7984 2 回目 v 15970 f 31936

2 Catmull-Clark 細分割曲面

2.1 4-to-1 分割の新しい 4 つの面を submesh に作成するコード

```

1
2   for ( auto& fc : mesh_->faces() ) {
3       //
4       // ここに4-to-1 分割の新しいつの面を 4 submesh に作成するコードを追加
5       auto he0 = fc->halfedge(0);
6       auto he1 = fc->halfedge(1);
7       auto he2 = fc->halfedge(2);
8       auto he3 = fc->halfedge(3);
9
10      auto v0 = he0->vertex();
11      auto v1 = he1->vertex();
12      auto v2 = he2->vertex();
13      auto v3 = he3->vertex();
14
15      auto e0 = he0->edge();
16      auto e1 = he1->edge();
17      auto e2 = he2->edge();
18      auto e3 = he3->edge();
19
20      auto ev0 = even_[v0->id()];
21      auto ev1 = even_[v1->id()];
22      auto ev2 = even_[v2->id()];
23      auto ev3 = even_[v3->id()];
24
25      auto ed0 = edvt_[e0->id()];
26      auto ed1 = edvt_[e1->id()];
27      auto ed2 = edvt_[e2->id()];
28      auto ed3 = edvt_[e3->id()];
29
30      auto fv = fcvt_[fc->id()];
31
32      submesh_->addRectangle(ev0, ed0, fv, ed3);
33      submesh_->addRectangle(ed0, ev1, ed1, fv);
34      submesh_->addRectangle(fv, ed1, ev2, ed2);
35      submesh_->addRectangle(ed3, fv, ed2, ev3);
36
37      //
38

```

```
39     }
40   };
```

2.2 プログラムの説明

Catmull-Clark 細分割において、元のメッシュの 1 つの四角形の面を、4 つの小さな四角形に分割し、新しいサブメッシュに登録する処理である。Loop 細分割と同様に、面のハーフエッジを取得して、頂点とエッジを取得する。頂点とエッジの間の点に加えて中央の点のポインタを取得して、反時計回りに頂点を選んで、4 つの四角形を作成した。

2.3 even vertex の頂点位置 p の計算コード

```
1
2   // even vertex point
3   for ( auto& vt : mesh->vertices() ) {
4       Eigen::Vector3d p;
5       //
6       // ここにeven vertex の頂点位置 $p$  の計算コードを追加
7       Eigen::Vector3d v = vt->point();
8
9       Eigen::Vector3d sum_F = Eigen::Vector3d::Zero();
10      Eigen::Vector3d sum_E = Eigen::Vector3d::Zero();
11      int n = 0;
12
13      VertexLCirculator vcirc(vt);
14      for (auto& he : vcirc.halfedges()) {
15          Eigen::Vector3d neighbor_v = he->next()->vertex()->point();
16          sum_E += (v + neighbor_v) * 0.5;
17
18          if (he->face()) {
19              Eigen::Vector3d f_point = Eigen::Vector3d::Zero();
20              f_point += he->face()->halfedge(0)->vertex()->point();
21              f_point += he->face()->halfedge(1)->vertex()->point();
22              f_point += he->face()->halfedge(2)->vertex()->point();
23              f_point += he->face()->halfedge(3)->vertex()->point();
24              f_point *= 0.25;
25
26              sum_F += f_point;
27          }
28      }
```

```

28         n++;
29     }
30     Eigen::Vector3d F = sum_F / (double)n;
31     Eigen::Vector3d E = sum_E / (double)n;
32
33     p = (F + 2.0 * E + (double)(n - 3) * v) / (double)n;
34
35     //
36
37     even_[vt->id()]->setPoint(p);
38 }

```

2.4 プログラムの説明

既存の頂点が細分割後に移動する新しい座標を計算し更新する処理である。VertexLCirculator を用いて対象頂点の周囲のハーフエッジを巡回し、接続するエッジの中点と、隣接する面の面頂点の座標をそれぞれ集計した。この際未計算のデータにアクセスする依存バグを防ぐため、面頂点は他の計算結果を参照せず、その場で面を構成する4頂点の平均として算出した。最後に集計から得られたエッジ中点の平均値 E、面頂点の平均値 F、頂点の次数 n を用い、Catmull-Clark の更新公式で計算し元の頂点の座標を更新した。

2.5 edge vertex の頂点位置 p の計算コード

```

1
2     // edge vertex point
3     for ( auto& ed : mesh_->edges() ) {
4         Eigen::Vector3d p;
5         //
6         // ここに edge vertex の頂点位置 p の計算コードを追加
7         auto he1 = ed->lhe();
8         auto he2 = ed->rhe();
9
10        Eigen::Vector3d v0 = he1->vertex()->point();
11        Eigen::Vector3d v1 = he2->vertex()->point();
12
13        Eigen::Vector3d f0 = Eigen::Vector3d::Zero();
14        f0 += he1->face()->halfedge(0)->vertex()->point();
15        f0 += he1->face()->halfedge(1)->vertex()->point();

```

```

16     f0 += he1->face()->halfedge(2)->vertex()->point();
17     f0 += he1->face()->halfedge(3)->vertex()->point();
18     f0 *= 0.25;
19
20     Eigen::Vector3d f1 = Eigen::Vector3d::Zero();
21     f1 += he2->face()->halfedge(0)->vertex()->point();
22     f1 += he2->face()->halfedge(1)->vertex()->point();
23     f1 += he2->face()->halfedge(2)->vertex()->point();
24     f1 += he2->face()->halfedge(3)->vertex()->point();
25     f1 *= 0.25;
26
27     p = (v0 + v1 + f0 + f1) * 0.25;
28
29     //
30
31     edvt_[ed->id()]->setPoint(p);
32 }

```

エッジ上に新しく生成される頂点の座標を計算する処理である。対象のエッジから左右方向のハーフエッジを取得し、端点の座標を出した。隣接する2つの面の面頂点の座標を、それぞれの面を構成する4頂点の平均としてその場で独自に算出した。そして更新式を計算して、新しいエッジ頂点の座標にした。

2.6 プログラムの説明

2.7 ace vertex の頂点位置 p の計算コード

```

1  for ( auto& fc : mesh_->faces() ) {
2      Eigen::Vector3d p;
3      //
4      // ここに face vertex の頂点位置  $p$  の計算コードを追加
5      auto he0 = fc->halfedge(0);
6      auto he1 = fc->halfedge(1);
7      auto he2 = fc->halfedge(2);
8      auto he3 = fc->halfedge(3);
9
10     Eigen::Vector3d v0 = he0->vertex()->point();
11     Eigen::Vector3d v1 = he1->vertex()->point();
12     Eigen::Vector3d v2 = he2->vertex()->point();

```



```

13     Eigen::Vector3d v3 = he3->vertex()->point();
14     p = (v0 + v1 + v2 + v3) * 0.25;
15
16     //
17
18     fcvt_[fc->id()]->setPoint(p);
19 }

```

2.8 プログラムの説明

面の中心に新しく生成される面頂点の座標を計算する処理である。面のハーフエッジを取得して、そこから四角形の4つの頂点の座標を取得して、あとはそれらの平均を計算して新しい面の中央の頂点の座標にした。

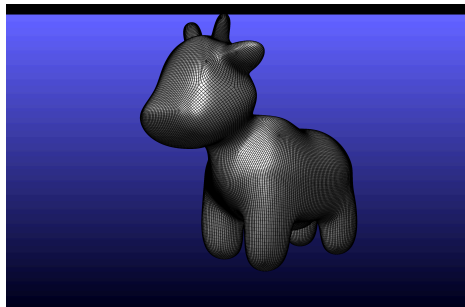


図3 図の説明

1回および2回細分割した後の頂点数・面数: 1回目 v 11714 f 11712 2回目 v 46850 f 46848

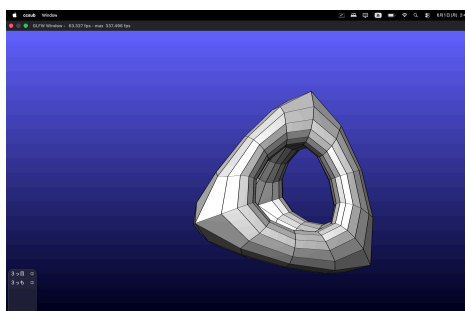


図4 図の説明

1回および2回細分割した後の頂点数・面数: 1回目 v 92 f 96 2回目 v 380 f 384